SOFTWARE REQUIREMENTS SPECIFICATION

Ledger — Distributed Task Queue and Job Scheduler

Field	Details
Project	Ledger
Document	Software Requirements Specification (SRS)
Version	1.0
Prepared by	Urmila Tamang
Student ID	2401538
Program	Information Technology — Dongshin University
Date	October 2026

Purpose: Define the functional, non-functional, interface, data, reliability, security, testing, and acceptance requirements for Ledger.

Table of Contents

No.	Section
1	Introduction and Project Overview
2	Purpose, Goals and Scope
3	System Context and Users
4	Context Diagram
5	Functional Requirements
6	Job Lifecycle and Business Rules
7	External Interfaces and API Requirements
8	Data Requirements and Database Design
9	Reliability, Concurrency and Failure Recovery
10	Non-Functional Requirements
11	Security Requirements
12	Dashboard and UI Requirements
13	Technology and Architecture
14	Testing and Verification
15	Acceptance Criteria
16	Out of Scope and Future Enhancements
17	Conclusion

Traceability: Requirement IDs are used so that implementation and tests can be mapped back to this SRS.

1. Introduction and Project Overview

1.1 Project Description

Ledger is a distributed task queue and job scheduler designed to execute background jobs reliably across multiple worker processes or machines. An application submits work to Ledger; Ledger persists the job, queues it, assigns it to an available worker, records the result, and handles retries or recovery when failures occur.

1.2 Problem Statement

A basic queue does not fully address worker crashes, competing workers, temporary failures, delayed work, recurring schedules, or operational monitoring. Ledger addresses these concerns through persistent state, atomic job claiming, leases, retries, scheduling, dead-letter handling, logs, and metrics.

1.3 Objectives

• Accept jobs through an API.

• Persist job state and execution history.

• Process jobs concurrently with multiple workers.

• Recover work after worker failure without silent loss.

• Support priorities, delayed jobs, recurring schedules, retries, and dead-letter queues.

• Provide a dashboard, logs, and metrics for operational visibility.

• Demonstrate distributed-systems concepts such as leases, heartbeats, locking, idempotency, and horizontal scaling.

1.4 Intended Audience

The project team, instructor/evaluator, developers, testers, administrators, and reviewers of the system design.

2. Purpose, Goals and Scope

2.1 Primary Goals

• Reliable asynchronous execution.

• Concurrent worker processing.

• Persistent job lifecycle and history.

• Configurable retries and failure recovery.

• Delayed and recurring scheduling.

• Queue/worker monitoring.

• Safe coordination of concurrent workers.

2.2 In Scope

Area	Capabilities
Job management	Submit, inspect, cancel, retry, delete, schedule, and view history.
Queue management	Multiple queues, priorities, pause/resume, statistics, depth monitoring.
Worker management	Registration, heartbeats, health, assignment, failure detection.
Reliability	Acknowledgements, leases, retries, backoff, max attempts, recovery, dead-letter queues.
Scheduling	Delayed, one-time, and cron-like recurring jobs.
Persistence	PostgreSQL storage for jobs, attempts, queues, schedules, workers, and history.
Monitoring	Dashboard, history, queue metrics, worker metrics, failures, structured logs.

2.3 Out of Scope

• Full workflow/DAG orchestration.

• Distributed transactions and multi-region consensus.

• Kubernetes-native scheduling and container orchestration.

• GPU-specific scheduling.

• Machine-learning-specific scheduling.

• Full enterprise RBAC in the first release.

3. System Context and Users

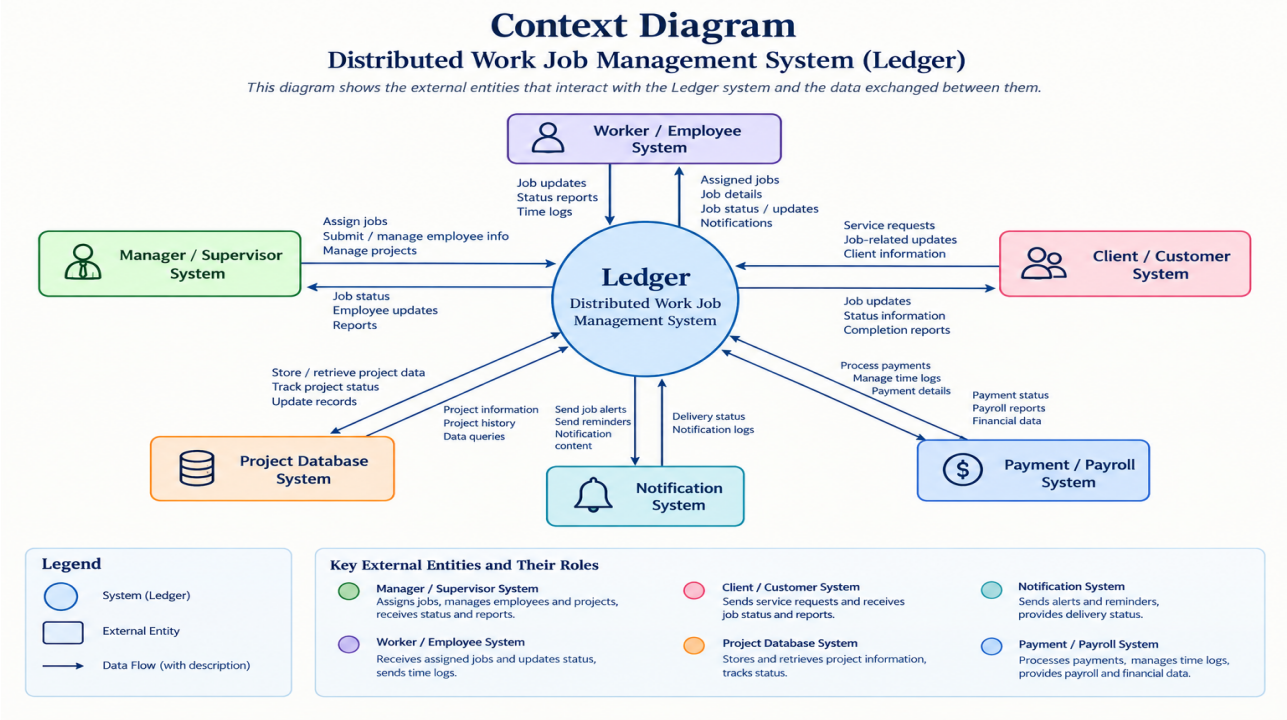
Actor	Role
Application / Client	Uses API/SDK to submit, query, schedule, cancel, and retry jobs.
Worker	Registers, receives jobs, executes them, reports success/failure, and sends heartbeats.
Administrator / Supervisor	Monitors jobs, queues, workers, schedules, failures, and dead-letter jobs.
PostgreSQL	Durable source of truth for job state, attempts, schedules, queues, workers, and history.

3.1 System Boundary

Ledger is the central system of interest. External clients, workers, administrators, and supporting infrastructure communicate with Ledger through defined interfaces. Internal implementation details are described later in the architecture section.

4. Context Diagram

The context diagram shows Ledger as the central system and identifies the major external entities and data flows. It is a high-level system-boundary view, not an internal process diagram.



Key Data Flows

• Manager/Supervisor → Ledger: job assignment, project and employee information.

• Ledger → Manager/Supervisor: status, updates, and reports.

• Worker → Ledger: job updates, execution results, time logs, and heartbeats.

• Ledger → Worker: assigned jobs, job details, and status information.

• Client → Ledger: service/job requests and related information.

• Ledger → Client: job updates, status, and completion reports.

• Ledger ↔ PostgreSQL: durable project/job state, history, schedules, and worker data.

• Ledger → Notification System: alerts and reminders.

• Ledger ↔ Payment/Payroll where applicable: time, payment, and financial status data.

5. Functional Requirements

The following requirements define what the system shall do. Each requirement is intended to be testable.

ID	Requirement
FR-001	Job Submission — accept job type, payload, queue, and priority and return a unique job ID.
FR-002	Unique Job IDs — use globally unique identifiers for status, cancellation, retry, history, and logs.
FR-003	Multiple Queues — support multiple logical queues.
FR-004	Queue Isolation — workers may subscribe to one or more queues and receive eligible jobs.
FR-005	Priority — select higher-priority eligible jobs first while documenting fairness.
FR-006	Worker Registration — register ID, hostname, process ID, queues/types, concurrency, and startup time.
FR-007	Job Acquisition — workers request jobs and Ledger atomically assigns eligible work.
FR-008	Job Lease — reserve claimed jobs with worker identity and lease expiration.
FR-009	Acknowledgement — accept success acknowledgement or failure report.
FR-010	Retry — retry failed jobs according to configurable attempts and delay policy.
FR-011	Exponential Backoff — support exponential retry delay with a maximum.
FR-012	Dead-Letter Queue — move exhausted jobs to DEAD_LETTER for inspection/retry.
FR-013	Cancellation — allow authorized cancellation of eligible jobs.
FR-014	Delayed Jobs — support jobs that become eligible after a specified time.
FR-015	One-Time Scheduling — schedule a job for a future time.
FR-016	Recurring Scheduling — support cron-like recurring schedules.
FR-017	Schedule Control — create, view, update, enable, disable, and delete schedules.
FR-018	Worker Heartbeats — periodically record worker heartbeats.
FR-019	Worker Health — expose worker status, utilization, heartbeat time, and failures.
FR-020	Job History — retain attempts, timestamps, worker identity, duration, and errors.
FR-021	Queue Monitoring — expose depth, waiting/running counts, and processing statistics.
FR-022	Dashboard — monitor jobs, workers, queues, schedules, and failures.
FR-023	Structured Logging — produce timestamped structured diagnostic logs.
FR-024	Metrics — expose job, queue, worker, scheduler, latency, and failure metrics.
FR-025	Idempotency — optionally support idempotency keys to reduce duplicate side effects.

6. Job Lifecycle and Business Rules

6.1 Job States

A job may move through CREATED, QUEUED, SCHEDULED, RUNNING, SUCCEEDED, FAILED, RETRYING, CANCELLED, and DEAD_LETTER. Invalid state transitions shall be rejected.

Normal: CREATED → QUEUED → RUNNING → SUCCEEDED
Failure: RUNNING → FAILED → RETRYING → QUEUED; if no retries remain, FAILED → DEAD_LETTER

6.2 Queue Selection

Eligible jobs should be selected using queue membership, availability time, priority, creation time, and fairness. Concurrent workers must not unintentionally claim the same job.

6.3 Retry Rules

• Increment attempt count after failure.

• If attempts remain, reschedule using configured delay/backoff.

• If attempts are exhausted, enter DEAD_LETTER.

• Retain each execution attempt in history.

6.4 At-Least-Once Delivery

Ledger targets at-least-once delivery. A job is not silently lost, but a worker failure between execution and acknowledgement can cause re-execution. Handlers should therefore be idempotent when duplicate effects are harmful.

7. External Interfaces and API Requirements

7.1 Core API

Method	Endpoint	Purpose
POST	/api/jobs	Submit job.
GET	/api/jobs	List/filter jobs.
GET	/api/jobs/:id	Get job details/status.
POST	/api/jobs/:id/retry	Retry permitted failed/dead-letter job.
POST	/api/jobs/:id/cancel	Cancel a permitted job.
DELETE	/api/jobs/:id	Delete according to retention rules.
GET	/api/queues	List queues.
GET	/api/queues/:id	Get queue details.
POST	/api/queues	Create queue.
POST	/api/queues/:id/pause	Pause queue.
POST	/api/queues/:id/resume	Resume queue.
GET	/api/schedules	List schedules.
POST	/api/schedules	Create schedule.
GET	/api/schedules/:id	Get schedule.
PATCH	/api/schedules/:id	Update schedule.
DELETE	/api/schedules/:id	Delete schedule.
POST	/api/schedules/:id/enable	Enable schedule.
POST	/api/schedules/:id/disable	Disable schedule.
GET	/api/workers	List workers.
GET	/api/workers/:id	Get worker details.

7.2 Worker Protocol

Worker communication shall support REGISTER, JOB assignment, ACK, FAIL, and HEARTBEAT messages. HTTP, WebSockets, gRPC, or another documented protocol may be used, provided authentication and reliable error handling are implemented.

8. Data Requirements and Database Design

8.1 Persistence

PostgreSQL shall be the durable source of truth for job state and execution history. Persistent state must survive application and worker restarts.

8.2 Core Entities

Entity	Important Attributes
jobs	id, queue_id, job_type, payload, priority, status, attempts, max_attempts, scheduled_at, available_at, timestamps, last_error
job_attempts	id, job_id, attempt_number, worker_id, status, start/end time, duration, error details
queues	id, name, paused, created_at, updated_at
schedules	id, name, job_type, queue_id, cron_expression, payload, priority, enabled, next_run_at, last_run_at
workers	id, worker_name, hostname, process_id, status, concurrency, last_heartbeat, start/stop timestamps

8.3 Integrity Rules

• Job IDs are unique.

• A job belongs to one queue at a time.

• Invalid state transitions are rejected.

• Attempts reference existing jobs.

• Heartbeats are timestamped.

• Schedule generation avoids unintended duplicates.

• Transactions/row locking protect concurrent claims.

9. Reliability, Concurrency and Failure Recovery

9.1 Worker Coordination

Multiple workers may access the same queue. Job claiming shall be atomic. PostgreSQL row-level locking such as SELECT ... FOR UPDATE SKIP LOCKED may be used to let workers claim different jobs concurrently.

9.2 Failure Scenarios

• Worker crash → lease expiry → job becomes recoverable.

• Network disconnect → missed heartbeat/lease expiry → recovery.

• Database restart → persistent state remains available after recovery.

• Job timeout/failure → retry policy or dead-letter handling.

• Scheduler crash → schedules recover from persistent state without unintended duplicates.

• Server restart → queued and scheduled work remains recoverable.

9.3 Concurrency Requirements

• Do not normally claim one queued job simultaneously with multiple workers.

• Allow independent jobs to execute concurrently.

• Avoid unacceptable database lock contention.

• Adding workers should increase processing capacity.

10. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	Job submission target <100 ms p95 in the test environment.
NFR-002	Performance	Job acquisition target <100 ms p95.
NFR-003	Performance	Dashboard API target <300 ms p95.
NFR-004	Throughput	1,000+ jobs/minute initial engineering target in a controlled test environment.
NFR-005	Concurrency	100+ concurrent workers initial engineering target.
NFR-006	Heartbeat	Workers send heartbeats approximately every 5–10 seconds.
NFR-007	Failure detection	Worker failure normally detected within 30 seconds.
NFR-008	Scalability	API instances and workers can scale horizontally.
NFR-009	Availability	Transient failures do not silently lose persisted jobs.
NFR-010	Maintainability	Components have clear responsibilities and documented interfaces.
NFR-011	Observability	Logs and metrics support diagnosis of job/worker/queue/scheduler behavior.
NFR-012	Portability	Runs consistently in local and containerized environments.
NFR-013	Usability	Dashboard is understandable without raw database inspection.
NFR-014	Data Integrity	State/history remain consistent under concurrent operations.

11. Security Requirements

• Require API authentication for protected operations.

• Authenticate workers before job exchange.

• Restrict access through authorization rules.

• Do not hard-code or expose credentials/secrets in logs.

• Validate all job submissions.

• Treat job payloads as untrusted data.

• Apply rate limiting where appropriate.

• Avoid unnecessary sensitive payload data in logs.

• Use secure transport in deployed environments.

12. Dashboard and UI Requirements

12.1 Main Dashboard

• Show counts for Jobs, Workers, Queues, and Schedules.

• Show Success, Failed, Running, and Queued statistics.

• Show per-queue waiting and running counts.

• Provide navigation to jobs, workers, queues, schedules, and failures.

12.2 Job Detail

• Job ID, type, queue, priority, status, attempts.

• Created/started/finished timestamps.

• Executing worker.

• Latest error.

• Retry/Cancel actions when permitted.

• Execution history.

12.3 Monitoring Views

• Worker health and heartbeat status.

• Queue depth and oldest waiting job.

• Failed and dead-letter jobs.

• Schedule status and next execution time.

13. Technology and Architecture

Component	Recommendation
Backend	Go is recommended for concurrency, networking, and deployment; Java, Python, Rust, or Node.js are viable alternatives.
Database	PostgreSQL for durable job state, history, scheduling, queue metadata, and worker metadata.
Queue	Initial implementation can use PostgreSQL-based job claiming; Redis/RabbitMQ may be future alternatives.
Dashboard	Web-based administrative monitoring dashboard.
Deployment	Docker/docker-compose for reproducible development and testing.
Testing	Unit, integration, concurrency, failure-recovery, security, and load testing.

13.1 Logical Architecture

Client/Application → Ledger API → PostgreSQL + Scheduler + Dashboard → Job Queue → Worker Pool → Job Execution

The architecture separates submission from execution. API instances and workers can scale independently, while PostgreSQL maintains durable state. Scheduler instances create due jobs and use coordination to avoid duplicate generation.

14. Testing and Verification Requirements

Test Type	Verification Focus
Unit	State transitions, retry calculation, priority ordering, cron parsing, queue selection, lease expiration, heartbeat logic, idempotency.
Integration	API + PostgreSQL, worker + server, scheduler + database, queue operations, dashboard data.
Concurrency	Multiple workers competing for the same queue; safe atomic claiming and acceptable lock contention.
Failure Recovery	Worker crash, network disconnect, database restart, job timeout/failure, scheduler/server restart.
Load	Increasing job volume and worker count; throughput, average/p95 latency, DB usage, utilization, failure rate.
Security	Authentication, authorization, validation, rate limiting, secret handling, unsafe payload cases.

14.1 Verification Rule

A requirement is verified when a repeatable test or inspection demonstrates that the specified behavior is implemented and does not violate relevant reliability, security, or data-integrity rules.

15. Acceptance Criteria

Area	Acceptance Condition
Job Processing	Applications submit jobs; jobs persist; workers retrieve/execute them; successful jobs become SUCCEEDED.
Reliability	Failed jobs retry; limits are respected; exhausted jobs become DEAD_LETTER; leased jobs recover after worker failure.
Concurrency	Workers process concurrently; jobs are not normally double-claimed; more workers increase throughput.
Scheduling	Delayed, one-time, and recurring jobs execute correctly without unintended duplicate generation.
Monitoring	Administrators view status/history, worker health, queue depth, failures, and dead-letter jobs.
Persistence	State/history survive restarts and recoverable work is not silently lost.

16. Out of Scope and Future Enhancements

Future releases may add workflow/DAG orchestration, enterprise RBAC, Kubernetes-native scheduling, container orchestration, GPU-aware scheduling, multi-region deployment, richer SDKs, and advanced analytics. These should be considered only after the core reliability, persistence, concurrency, scheduling, monitoring, and testing requirements are stable.

17. Conclusion

Ledger is specified as a persistent distributed background-job execution platform rather than a simple queue. Its central engineering value is reliable asynchronous execution when multiple workers operate concurrently and infrastructure can fail. This SRS establishes a traceable foundation for implementation, testing, evaluation, and future extension.

Core concepts demonstrated: at-least-once delivery, job leasing, worker coordination, database locking, failure detection, heartbeats, retries, exponential backoff, dead-letter queues, idempotency, priority scheduling, cron scheduling, persistence, horizontal scaling, and observability.

End of Software Requirements Specification